

Coleções em JavaScript

Arrays, Objetos & Manipulação

Curso: Programador de Sistemas
Senac Centro | Noite



Roteiro da Aula

1. O que são Coleções de Dados?
2. Arrays (Vetores): Acesso reverso (.at) e manipulação.
3. Buscas em Arrays (includes).
4. Objetos Literais (Dicionários).
5. Arrays de Objetos & Iteração.



Checklist de Conceitos

- **Array (`[]`)**: Lista ordenada de itens indexados de 0 a N.
- **Acesso Reverso (`.at`)**: Acessar elementos de trás para frente usando índices negativos (ex: `.at(-1)`).
- **Modificadores**: `.push()` , `.pop()` , `.unshift()` , `.shift()` e `.splice()` .
- **Buscas**: Verificar a presença com `.includes()` .
- **Objeto (`{}`)**: Coleção chave-valor para descrever um item único.
- **Iteração (`for...of`)**: Como ler e processar coleções automaticamente.

Arrays: Listas de Dados

Um array armazena vários elementos de forma ordenada na memória.

```
let alunos = ["Ana", "Bruno", "Carlos"];

// Acesso tradicional (por colchetes):
console.log(alunos[0]); // "Ana"

// Acesso alternativo (com o método .at):
console.log(alunos.at(0)); // "Ana"
```

Visualizando o Array na Memória

Cada item fica associado a um índice inteiro fixo, iniciando em 0:

Índice	0	1	2
Valor	"Ana"	"Bruno"	"Carlos"

- **Acesso Reverso (.at):**

- `alunos.at(-1)` → "Carlos" (último item)
- `alunos.at(-2)` → "Bruno" (penúltimo item)

🔺 Descobrindo o Tamanho com `.length`

A propriedade `.length` retorna a quantidade total de elementos do Array. Ela é atualizada dinamicamente conforme a lista muda.

```
let nomes = ["Ana", "Bruno"];  
console.log(nomes.length); // 2  
  
nomes.push("Carlos");  
console.log(nomes.length); // 3 (Atualizado!)
```

Manipulando as Extremidades (Fim/Início)

Podemos adicionar ou remover elementos das pontas do Array:

- `.push()` / `.pop()` : Adiciona / remove da cauda (fim).
- `.unshift()` / `.shift()` : Adiciona / remove da cabeça (início).

```
let frutas = ["Uva"];  
frutas.push("Laranja"); // ["Uva", "Laranja"]  
frutas.unshift("Maçã"); // ["Maçã", "Uva", "Laranja"]
```



Pilhas vs Filas na Prática

- **Pilha (Descarte de Cartas):** Entra e sai pelo mesmo lado (topo).

```
let cartas = ["Ás", "Valete"];  
cartas.push("7 de Ouros"); // Adiciona no topo  
let topo = cartas.pop(); // Remove "7 de Ouros"
```

- **Fila (Fila do Spotify):** Entra no fim, sai pelo início.

```
let musicas = ["Música A", "Música B"];  
musicas.push("Música C"); // Entra na fila  
let proxima = musicas.shift(); // Toca "Música A"
```


Manipulando Posições Específicas

Para inserir ou remover itens em **qualquer posição intermediária**, usamos o método `.splice(indice_partida, quantidade_deletar, novos_itens...)` :

```
let cores = ["azul", "verde", "amarelo"];

// 1. Remover 1 item a partir do índice 1 (remove "verde"):
cores.splice(1, 1); // ["azul", "amarelo"]

// 2. Inserir "vermelho" no índice 1 sem deletar nada:
cores.splice(1, 0, "vermelho"); // ["azul", "vermelho", "amarelo"]
```



Existe esse item na Lista?

No Python, você usa `if item in lista`. No JavaScript, não use a palavra `in` para isso (ela busca índices). Em vez disso, use `.includes(item)`:

```
let convidados = ["Ana", "Bruno"];

if (convidados.includes("Ana")) {
  console.log("Ana está convidada!"); // Executa!
}
```

🪜 Percorrendo Listas com `for...of`

O loop `for...of` lê sequencialmente cada elemento da lista:

```
let nomes = ["Ana", "Bruno", "Carlos"];

for (let nome of nomes) {
  console.log("Aluno:", nome.toUpperCase());
}
```

Prática 1: Hora de Codificar!

- Abra o arquivo `exercicios_colecoes.js` no seu editor.
- Resolva os **EXERCÍCIOS 1, 2 e 3** (Manipulação de Arrays).
- Teste executando no terminal Node.js!



Objetos: Dicionários Chave-Valor

Um **Objeto Literal** representa um único registro complexo. Ele usa chaves nomeadas em vez de índices.

```
let produto = {  
  nome: "Notebook",  
  preco: 3200.00,  
  emEstoque: true  
};  
  
console.log(produto.nome); // "Notebook" (Acesso por ponto)
```



Modificando Objetos Dinamicamente

Chaves e valores no JS podem ser alterados ou adicionados livremente após o objeto ter sido criado:

```
let cliente = { nome: "Maria" };  
  
// 1. Alterando valor existente:  
cliente.nome = "Maria Silva";  
  
// 2. Criando nova propriedade no voo:  
cliente.pontos = 150;
```

Prática 2: Hora de Codificar!

- No mesmo arquivo `exercicios_colecoes.js`, resolva os EXERCÍCIOS 4, 5 e 6 (Objetos Literais).
- Valide os resultados exibidos no terminal!



Array de Objetos (Tabelas de Dados)

Na vida real, combinamos arrays e objetos para formar tabelas estruturadas:

```
let carrinho = [  
  { produto: "Fone", valor: 150 },  
  { produto: "Cabo HDMI", valor: 45 }  
];  
  
console.log(carrinho[0].produto); // "Fone"
```


🪑 Loops em Array de Objetos

Podemos iterar o array e acessar os atributos individuais de cada objeto da lista:

```
let estoque = [  
  { nome: "Teclado", preco: 250 },  
  { nome: "Mouse", preco: 120 }  
];  
  
for (let item of estoque) {  
  console.log(`Produto: ${item.nome} | R$ ${item.preco}`);  
}
```

Desafio Final: Sua Vez!

- No mesmo arquivo `exercicios_colecoes.js`, resolva os EXERCÍCIOS 7, 8, 9 e 10 (Integração e Loops).
- Teste os diferentes cenários no terminal do Node.js!

Conclusão

Agora você sabe armazenar e manipular múltiplos dados de forma flexível no JavaScript!

Próxima Aula: Introdução ao HTML5 e CSS3 (Fundamentos Web).